Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ижевский государственный технический университет
имени М.Т. Калашникова»
(ИжГТУ имени М.Т. Калашникова)
Факультет «Математика и естественные науки»
Кафедра «Прикладная математика и информационные технологии»

Практическая работа № 2
по курсу **«Методы и системы обработки неструктурированных данных»**
Вариант 6

Направление подготовки 01.03.02 «Прикладная математика»
Профиль – «Прикладной анализ данных и искусственный интеллект»

Проверил: В. Г. Суфиянов
д.т.н., профессор

Выполнил: М. А. Панин
студент группы Б23-183-1

Ижевск - 2026

СОДЕРЖАНИЕ

Содержание

Введение 3
Задание 1. Импорт библиотек и загрузка данных 3
Задание 2. Реализация конвейера предобработки 3
Задание 3. Построение матрицы CountVectorizer 4
Задание 4. Построение TF-IDF матрицы 5
Задание 5. Анализ разреженности матрицы 5
Задание 6. Форматы хранения разреженных матриц 6
Задание 7. Исследование влияния параметров 6
Задание 8. Индивидуальное задание (вариант 6) 8
1. Предобработка (удаление стоп-слов и лемматизация) 8
2. Словарь 8
3. DF и IDF 9
4. TF и TF-IDF для документа 3 9
5. Частотная матрица и плотность 9
6. CSC-представление матрицы 10
Заключение 10

Введение

Цель работы — освоить методы построения признакового пространства для коллекции текстовых документов: реализовать конвейер лингвистической предобработки, построить матрицы частот (CountVectorizer) и TF-IDF (TfidfVectorizer), проанализировать разреженность и сравнить форматы хранения разреженных матриц.

Задание 1. Импорт библиотек и загрузка данных

Выполнен импорт стандартных библиотек, средств предобработки и векторизации. Корпус новостей Lenta.Ru загружен из файла `/Users/michael/Documents/study6/nlp/datasets/lenta-ru-news.csv` и случайным образом отобрано 10 000 документов для обработки (ограничение не более 100 тыс.).

Листинг 2.1 — Импорт библиотек и загрузка данных

```
import numpy as np
import pandas as pd
import re
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer
import pymorphy2
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

data_path = "/Users/michael/Documents/study6/nlp/datasets/lenta-ru-news.csv"
df = pd.read_csv(data_path)
df_sample = df.sample(n=100000, random_state=42)
documents = df_sample["text"].astype(str).tolist()
```

Задание 2. Реализация конвейера предобработки

Конвейер включает очистку текста, токенизацию, удаление стоп-слов и нормализацию токенов (лемматизация либо стемминг). Для векторизаторов токены объединяются в строку.

Листинг 2.2 — Конвейер предобработки

```
class TextPreprocessor:
    def __init__(self, language="russian", use_lemmatization=True):
        self.language = language
        self.use_lemmatization = use_lemmatization
        self.stop_words = set(stopwords.words(language))
        if use_lemmatization:
            self.lemmatizer = pymorphy2.MorphAnalyzer()
        else:
            self.stemmer = SnowballStemmer(language)
```

```python

    def clean_text(self, text):
        if not isinstance(text, str):
            return ""
        text = text.lower()
        text = re.sub(r"<[^>]+>", " ", text)
        text = re.sub(r"https?://\S+|www\.\S+", " ", text)
        text = re.sub(r"\S+@\S+", " ", text)
        text = re.sub(r"\d+", " ", text)
        text = re.sub(r"[^\w\s]", " ", text)
        text = re.sub(r"[^a-яёа-z\s]", " ", text)
        text = re.sub(r"\s+", " ", text)
        return text.strip()

    def tokenize(self, text):
        return word_tokenize(text, language="russian")

    def normalize_token(self, token):
        if self.use_lemmatization:
            return self.lemmatizer.parse(token)[0].normal_form
        return self.stemmer.stem(token)

    def preprocess_for_vectorizer(self, text):
        cleaned = self.clean_text(text)
        tokens = self.tokenize(cleaned)
        processed = []
        for token in tokens:
            if token not in self.stop_words and len(token) > 2:
                processed.append(self.normalize_token(token))
        return " ".join(processed)
```

Задание 3. Построение матрицы CountVectorizer

Построена матрица «документ-термин» для разных параметров `CountVectorizer` и произведена оценка размерности, плотности и словаря.

Листинг 2.3 — Построение матрицы CountVectorizer

```python
vectorizer_base = CountVectorizer()
X_base = vectorizer_base.fit_transform(preprocessed_docs)

vectorizer_params = CountVectorizer(max_df=0.8, min_df=5, max_features=1000)
X_params = vectorizer_params.fit_transform(preprocessed_docs)
```

Пример результата (выборка 10 000 документов):

Таблица 3.1 — Характеристики CountVectorizer

Модель	Размерность	NNZ	Плотность, %
Базовый	10000 × 58996	979740	0.1661
С параметрами	10000 × 1000	530283	5.3028

Первые признаки (базовый): aaa, aac, aad, aap, aar, aargm, aasm, aba, abake, abassi. Первые признаки (с параметрами): associated, bbc, com, daily, facebook, news, press, reuters, the, times.

Задание 4. Построение TF-IDF матрицы

Построена TF-IDF матрица, изучено влияние параметров на словарь и плотность.

Листинг 2.4 — Построение TF-IDF матрицы

```
tfidf_base = TfidfVectorizer()
X_tfidf_base = tfidf_base.fit_transform(preprocessed_docs)

tfidf_params = TfidfVectorizer(
    max_df=0.8,
    min_df=5,
    max_features=1000,
    norm="l2",
    use_idf=True,
    smooth_idf=True,
    sublinear_tf=True,
)
X_tfidf_params = tfidf_params.fit_transform(preprocessed_docs)
```

Пример результата (выборка 10 000 документов):

Таблица 4.1 — Характеристики TF-IDF

Модель	Размерность	NNZ
Базовый	10000 × 58996	979740
С параметрами	10000 × 1000	530283

Первые признаки: associated, bbc, com, daily, facebook, news, press, reuters, the, times. Примеры idf: 4.43, 4.91, 4.97, 4.68, 4.88, 4.16, 4.38, 3.94, 3.44, 4.90.

Задание 5. Анализ разреженности матрицы

Для матриц CountVectorizer и TfidfVectorizer рассчитаны плотность и статистики по количеству признаков на документ.

Листинг 2.5 — Анализ разреженности

```
n_rows, n_cols = matrix.shape
n_nonzero = matrix.nnz
n_elements = n_rows * n_cols

density = n_nonzero / n_elements * 100
row_counts = np.array((matrix > 0).sum(axis=1)).flatten()
```

Таблица 5.1 — Статистика разреженности (выборка 10 000)

Матрица	Плотность, %	Среднее	Медиана	Макс
CountVectorizer (базовый)	0.1661	97.97	94	335
CountVectorizer (с параметрами)	5.3028	53.03	51	173
TfidfVectorizer (с параметрами)	5.3028	53.03	51	173

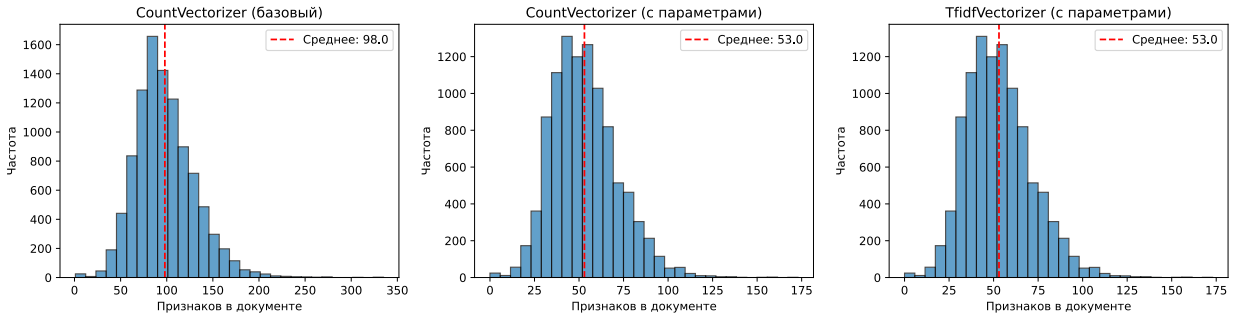

Рис. 1. **Рисунок 5.1 — Распределение числа признаков в документах**

Задание 6. Форматы хранения разреженных матриц

Произведено сравнение форматов COO, CSR, CSC и оценена экономия памяти по отношению к плотной матрице.

Листинг 2.6 — Сравнение форматов хранения

```
coo = coo_matrix((data, (rows, cols)), shape=(3, 5))
csr = coo.tocsr()
csc = coo.tocsc()
```

Задание 7. Исследование влияния параметров

Исследовано влияние `min_df` и `max_df` на размер словаря и плотность матрицы.

Таблица 7.1 — Эксперимент min_df (TF-IDF)

min_df	n_features	density, %	mean_features
1	58996	0.1661	97.97

2	28891	0.3287	94.96
5	14890	0.6130	91.28
10	9508	0.9230	87.76
20	5985	1.3864	82.98
50	3084	2.3983	73.96

Таблица 7.2 — Эксперимент max_df (TF-IDF)

max_df	**n_features**	**density, %**	**mean_features**
0.5	28888	0.3223	93.09
0.6	28889	0.3243	93.69
0.7	28891	0.3287	94.96
0.8	28891	0.3287	94.96
0.9	28891	0.3287	94.96
1.0	28891	0.3287	94.96

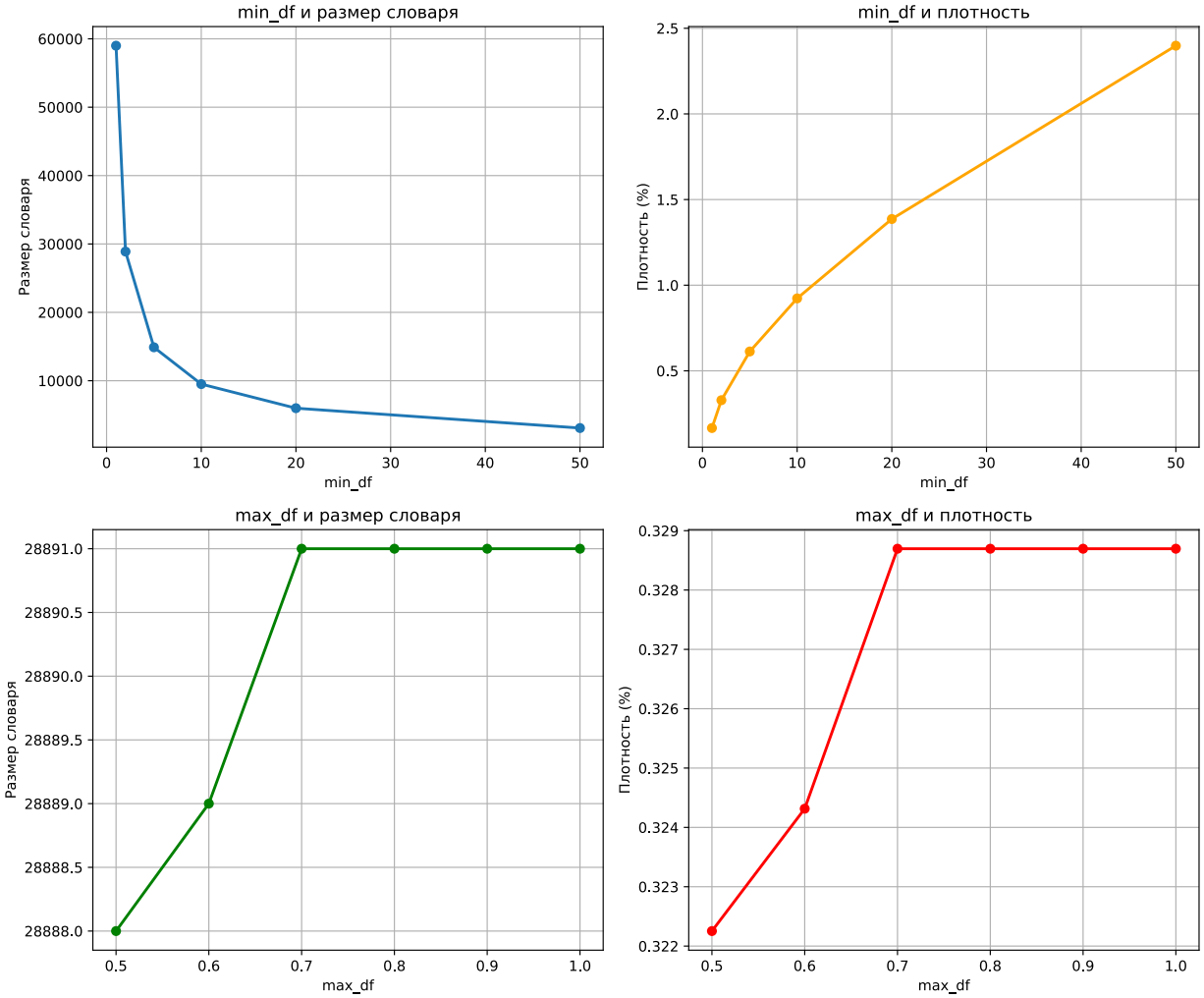

Рис. 2. **Рисунок 7.1 — Влияние min_df и max_df на размер словаря и плотность**

Задание 8. Индивидуальное задание (вариант 6)

1. Предобработка (удаление стоп-слов и лемматизация)

Корпус после обработки:

- Документ 1: сегодня, идти, дождь
- Документ 2: дождь, идти, второй, день
- Документ 3: завтра, дождь, быть

2. Словарь

Словарь (7 термов): сегодня, идти, дождь, второй, день, завтра, быть.

3. DF и IDF

Использовано $\mathrm{idf}(t) = \ln\left(\frac{N}{\mathrm{df}(t)}\right)$, $N = 3$.

Таблица 8.1 — DF и IDF

Терм	DF	IDF
сегодня	1	1.10
идти	2	0.41
дождь	3	0.00
второй	1	1.10
день	1	1.10
завтра	1	1.10
быть	1	1.10

4. TF и TF-IDF для документа 3

Документ 3 содержит 3 терма, поэтому $\mathrm{tf} = \frac{1}{3} = 0.33$.

Таблица 8.2 — TF и TF-IDF (документ 3)

Терм	TF	TF-IDF
завтра	0.33	0.37
дождь	0.33	0.00
быть	0.33	0.37

5. Частотная матрица и плотность

Порядок термов: сегодня, идти, дождь, второй, день, завтра, быть.

Таблица 8.3 — Частотная матрица

Док.	сегодня	идти	дождь	второй	день	завтра	быть
1	1	1	1	0	0	0	0
2	0	1	1	1	1	0	0
3	0	0	1	0	0	1	1

Всего элементов: $3 * 7 = 21$, ненулевых: 10, плотность: $\frac{10}{21} = 47.62\%$.

6. CSC-представление матрицы

Индексы строк — с нуля. Порядок столбцов соответствует словарю.

- `data`: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
- `row_ind`: [0, 0, 1, 0, 1, 2, 1, 1, 2, 2]
- `col_ptr`: [0, 1, 3, 6, 7, 8, 9, 10]

Заключение

В работе реализован конвейер предобработки текста, построены матрицы признаков CountVectorizer и TfidfVectorizer, выполнен анализ разреженности и рассмотрены форматы хранения разреженных матриц. Индивидуальное задание выполнено вручную с подробными вычислениями TF, IDF и TF-IDF.